An approach to software maintenance education

by B.J. Cornelius, M. Munro and D.J. Robson

The majority of courses in software engineering concentrate on educating students in the methods applicable to the development stage of the software lifecycle. Software maintenance is recognised as the most expensive phase of the software lifecycle, and yet it receives very little attention from those involved in software engineering education. This paper describes a novel approach to software maintenance education which is being undertaken at the University of Durham.

1 Background

In this paper, software maintenance is defined to be any work that is undertaken after delivery of a software system. It has been split into three subtasks: namely corrective, adaptive, and perfective maintenance (Ref. 1). Corrective maintenance is defined as that part of the maintenance effort which is devoted to the correction of errors. Adaptive maintenance tasks are the amendments required when the system is moved to a new machine or when changes are made to the external environment in which the system operates. Perfective maintenance is the enhancement of the system to incorporate new tasks which were not previously part of the requirements.

It has been recognised that software maintenance can consume as much as 70% of the costs during the lifecycle of a software system (Ref. 2). Thus the maintenance phase is a vast consumer of resources, both financial and human. The education of software engineers, however, has largely concentrated on techniques applicable to the development phase (Ref 3.), ignoring the work that will be undertaken in the future by a large number of the course graduates. This has paralleled the research activity in software engineering where there is a similar emphasis on the development as opposed to the maintenance phase (Ref. 4). It has been argued that the new development techniques will reduce maintenance costs in the future. Lientz and Swanson's survey (Ref. 2), however, also showed that 60% of the maintenance phase is taken up with perfective maintenance and only 20% is corrective maintenance. New development techniques may well reduce corrective maintenance, but have little effect on the most expensive subtask, perfective maintenance.

There has also been considerable investment in existing software. It is often not cost-effective to throw away existing software and rewrite it with the latest development technique. Thus software continues to evolve. The accompanying system documentation, if any, and requirements may or may not be updated as the program is modified and, if not, then this makes future maintenance more difficult. Thus the code is often regarded as the only reliable and up-to-date source of information as to the exact function and behaviour of the system.

Understanding the functions and behaviour of a system given just the code is a vital part of the maintenance engineer's task. However, the main emphasis in practical work in both computer science and software engineering courses is on developing new software. Having completed the practical work, it is usually never accessed again. Students are also rarely exposed to existing code developed by someone else.

Once students take up employment in industry, the situation is almost reversed. Software in the industrial environment is written to be utilised, not to be ignored upon completion. It has been shown that software must evolve (Ref. 5) to continue to be useful, and thus perfective maintenance work is a common activity in industry. To address these problems, the University of Durham includes software maintenance as part of its teaching of software engineering.

2 Course design

The University of Durham started teaching courses in software engineering in 1984. The courses have since developed and expanded so that they now form a substantial part of the curriculum and are taught in all three years of the degree courses. Accompanying the second year course has been practical work similar to that used at the University of Southampton (Ref. 6) which was originally based on the 'Software Hut' principle (Refs. 7, 8). The practical work includes the development of a large program by a team. Although this has worked extremely well, it does not encompass any aspects of software maintenance.

It was decided to develop a software maintenance course which would be divided into two parts. First, there would be a software maintenance workshop, and this would be followed by a lecture course on maintenance. The reason for the workshop preceding the lecture course was to attempt to illustrate some of the problems of software maintenance, before illustrating possible solutions, and to give the students motivation for the lecture course. This paper is solely concerned with the workshop, as we believe this course element is novel. This Section describes the original design ideas for the workshop, and the following Section discusses the actual implementation.

The overall objective of the workshop was to give the students experience of software maintenance work. They would be given a program of nontrivial size and be required to maintain it. They would also work in small groups rather than individually, to reflect the normal industrial organis-

ation. The groups would be presented with a list of change requests, and it was intended that this list would include all types of maintenance activity — corrective, adaptive and perfective.

There would be one-hour weekly meetings with the instructors involved with the workshop, but no formal lectures. The workshop was divided into the following phases:

- familiarisation with the system
- selection of enhancements
- design of selected enhancements
- reviews
- implementation of enhancements
- new function and regression testing.

The workshop was divided into two parts, with deliverables at the end of each part. The first deliverable would be the rationale and design of the enhancements. The second deliverable would be a single copy of the enhanced system and a report of the testing.

The initial phase was familiarisation with the system by running test programs through the binary version of the system. It was decided that the students would receive a printed version of the source code on the first day, but a machine-readable copy would not be made available until after the reviews. The rationale for this decision was to make the students read the code and design their enhancements without trying them out. As they would only have a certain amount of time for this workshop, they would have to decide which of the change requests was the most important and whether they could be implemented within the limited timescales. The first deliverable would contain a rationale for the choice of change requests. The team would then allocate the work between individuals and each student would design the changes to the code to satisfy the selected change requests.

In the week before the first deadline, students would undertake reviews, with each member of the team taking turns at the roles of chair, reviewer and reviewee. The workshop instructors would be present, but would not allocate any marks for the review. The reviews would follow the normal pattern of software reviews (Ref. 9), but would adopt some of the techniques of the active design review technique (Ref. 10). In particular, reviewees supplied reviewers with a questionnaire which forced the reviewers to study the sections of the code which were being modified. The questions would not be passive such as 'Will the required modification work?', but active such as 'What are the preconditions of the modified procedure?' The chair of each review committee would produce a report on the problems found in the review and the reviewee would produce an updated design document that would be reviewed again before commencing implementation. This two-step review was felt to be sufficient for this course.

During the remaining three weeks, the students would implement the selected change requests and produce a single version of the system, with the modified parts clearly commented with the initials of the person responsible for the modification. They would be expected to have thoroughly tested the system before submission. They would work as individuals during this phase, except when they needed to produce the final system or when some of their changes conflicted.

3 Course implementation

The students who would be taking the course were in the third term of their second year of an Honours degree in Computer Science. They had completed a course in software engineering covering requirements analysis, specification and design techniques, verification and validation methods and configuration management. In particular, they were familiar with reviewing techniques, but had little practical experience of such reviews. In parallel with this course, the students had undertaken a group-based software engineering project, and so they were familiar with the advantages and disadvantages of working within a group.

The program chosen as the object to be maintained was a compiler and interpreter for the language Pascal-S (Ref. 11), which is a subset of Pascal. The program was 4346 lines long and was originally developed some fifteen years ago. The program was well structured, but this appears to be the only design method employed and it makes a great deal of use of global variables. This system was supported by a

The students were given the following change requests. Each change request could not be subdivided:

- **Implement checks on for loop control variables. Check for threats, non-local and nonstructured variables.**
- **Implement the loop and exit construct of Modula-2.**
- **Implement an otherwise arm in a case statement, ranges for case labels and improve the efficiency of the code generation for a case statement.**
- **Improve the symbol table routines by implementing the routines as a binary tree or hash table.**
- **Implement string operations and a string data type as in Turbo Pascal or UCSD Pascal.**
- **Implement conformant array parameters as in Pascal.**
- **Implement pointers as in Pascal.**
- **Implement the goto statement as in Pascal.**
- **Improve the error message handling by having a file of compile-time and run-time error messages, suppression of undeclared identifier messages and improved error diagnostics.**

Fig. 1 Summary of change requests

- Exactly how is the Jump instruction at the end of the case table used?
- What information is logged about each compilation error and how is the information stored?
- In what order are the symbol table entries displayed?
- How do the proposed changes detect attempted exits within procedures and functions, and what error message is produced?
- In what other ways could a variable's usage as a control variable be indicated?

Fig. 2 Examples of questions given to reviewers

book (Ref. 12) which fully described the system and avoided the problem of students spending too long on the familiarisation phase or on mastering the compilation techniques employed (the students had not yet taken a course in compiler construction). The binary version contained a tracing mode and a debugging mode which enabled the students to follow the internal procedure calling within the system and to obtain dumps of the symbol table and the generated code. The supporting book contained a source listing of the system. It could be argued that providing the students with a fully documented, well-structured and well-behaved system was not exposing them to the realities of the outside world, but this compromise was necessary because of the time constraints.

The software maintenance workshop was spread over six weeks, and students would spend six hours per week on the workshop. During this period, the students were taking three other workshops of similar length. The deadline for the first deliverable was set to be the halfway point, with the second deliverable required at the end of the course. The students were given a number of change requests (see Fig. 1), some of which were infeasible within the time constraints. None of the change requests were minor. The time constraints also led to a decision by the instructors to limit the change request to those involving perfective maintenance. Perfective maintenance forms the largest part of the maintenance work, and so this was not expected to compromise the original aims of the workshop.

The reviews took place as intended at the end of the third week, but again, because of time constraints, it was decided not to have a second round of reviews, but only require that the students submit a revised design document. To illustrate the benefits of good design documents, the students implemented a design other than their own, the allocation being chosen at random.

The students were assessed on the basis of deliverables, namely the design documents and the final delivered version of the software. The instructors developed their own testing suite in order to evaluate the final product. Each student in the group was given an overall mark for the quality of the product, and this was adjusted on an individual basis after considering the individual design documents, the relative difficulty of the selected change requests and the quality of those changes.

4 Evaluation

The workshop took place in the Summer of 1988 with a class size of five students. Owing to the small numbers, they were formed into a single group. The familiarisation stage went smoothly, but the students did not explore the binary version very thoroughly. Unknown to the students, three errors were deliberately seeded in the binary version to see how well the students exercised the system, but none of them was detected. The machine-readable copy of the source code which was supplied did not contain these seeds, but the binary version that was generated from this source was not compared with the original binary version, and so the seeds remained undetected. This was probably a reflection that the students had been educated in a development environment, where they had written all the software and not performed any maintenance on that software. The students could be said to be too trusting in this respect.

There was some uncertainty by the students about which of the modifications to undertake, as there were some conflicting constraints, such as the desirability of some of the change requests, the limited amount of time available, the perceived difficulty of the change requests and the possible amount of marks available for different change requests. With hindsight, the instructors did not give this problem enough thought and a clearer strategy will have to be applied in future.

The reviews were particularly successful, revealing a number of errors in the proposed changes of all the students. All the final versions of the design documents were substantially different from those presented at the reviews. The reviews also revealed that the full impact of one of the changes had not been understood and that the suggested changes were insufficient. In the second phase of the workshop, this particular modification, the implementation of the symbol table as a binary tree, was redesigned rather than implemented. Examples of the questions asked of reviewers are illustrated (see Fig. 2). There were fears that some of the reviewees might interpret criticisms of their design as personal criticism and, although the instructors did flinch at some of the questions, this did not occur. This particular problem may arise when the workshop is undertaken by a larger group of students in future years.

Most of the modifications suggested in the design documents were inappropriate, but there were some errors largely due to some of the subtle programming language semantics of some of the change requests. For instance, the precise circumstances under which a control variable in Pascal can be said to be threatened were not fully appreciated.

The students were supplied with the machine-readable copy of the source code contained within a revision control system. To the disappointment of the instructors, the

revision control system was not used, and greater emphasis will be given to the benefits of such a system in future workshops.

5 Conclusions

The study of software maintenance is usually neglected in software engineering education. Too often students only get exposure to development techniques, with maintenance largely ignored. The aim of this workshop was to balance the students' education in software engineering and give the students experience of the type of activity usually undertaken during maintenance.

The workshop also introduced the students to a disciplined approach to software maintenance, where all the changes are fully documented and subject to peer review. The reviews were particularly successful and supported the instructors' view that group-based project work in software engineering is extremely valuable. The course was popular with the students and will be repeated with a much larger group next year.

6 Acknowledgments

The authors would like to acknowledge the assistance of Andrew Brabban, Russell Collingham, Yuet Ip, Nick Saville and Mike Smith who acted as guinea-pigs for this workshop.

7 References

1 SWANSON, E.F.: 'The dimensions of maintenance'. Proc. 2nd Int. Conf. on Software Engineering, IEEE, 1976, pp. 492–497

2 LIENTZ, B.P., and SWANSON, E.F.: 'Software maintenance management' (Addison-Wesley, 1980)

3 GIBBS, N.E., and FAIRLEY, R.E.: 'Software engineering education: the educational needs of the software community' (Springer-Verlag, 1987)

4 BENNETT, K.H., CORNELIUS, B.J., MUNRO, M., and ROBSON, D.J.: 'Software maintenance: a key area for research', *Univ. Comput.*, 1988, **10,** (4), pp. 184–188

5 LEHMAN, M.M., and BELADY, L.A.: 'A model of large program development', *IBM Syst. J.*, 1976, **15,** (3), pp. 225–252

6 GARRATT, P.W., and EDMUNDS, G.: 'Teaching software engineering at university', *Inf. Softw. Technol.*, 1988, **30,** (1), pp. 5–11

7 HORNING, J.J., and WORTMAN, D.B.: 'Software Hut: a computer program engineering project in the form of a game', *IEEE Trans. Softw. Eng.*, 1977, **SE-3,** (4), pp. 325–330

8 WORTMAN, D.B.: 'Software projects in an academic environment', *IEEE Trans. Softw. Eng.*, 1987, **SE-13,** (11), pp. 1176–1181

9 FAIRLEY, R.E.: 'Software engineering concepts' (McGraw-Hill, 1985)

10 PARNAS, D.L., and WEISS, D.: 'Active design reviews'. Proc. 8th Int. Conf. on Software Engineering, IEEE, 1986, pp. 132–136

11 WIRTH, N.: 'Pascal-S: a subset and its implementation', *in* BARRON, D.W. (Ed.): 'Pascal — the language and its implementation' (Wiley, 1981), pp. 199–260

12 REES, M.J., and ROBSON, D.J.: 'Practical compiling with Pascal-S' (Addison-Wesley, 1988)

The paper was first received on 10th August 1988 and in revised form on 28th February 1989.

The authors are with the Centre for Software Maintenance, University of Durham, Durham DH1 3LE, England.

COMPUTING BOOKS FROM THE IEE

Software Engineering 88

IEE Conference Publication number 290

Construction of software systems encompassing specification, design, production and modification, given uncertain and changing requirements • Measurement and quality assurance • Management issues affecting software • Beyond software engineering

200pp., 40 papers, 297 × 210 mm, soft covers
ISBN 0 85296 3650, 1988 **£36**

Software engineering public domain standards

This document lists publicly available standards applicable to the engineering of software systems. These standards are listed by identifier and title and in many cases with a short summary of their coverage. However they are listed without comment as to their appropriateness or quality since it is not the purpose of this catalogue to pass judgement or advise on best practice. Only British, American and International standards are covered.

16pp., 297 × 210mm, soft covers
ISBN 0 86341 152 5, 1988 **£15**

Software engineering for microprocessor systems

P. Depledge (Ed.)

Includes: Software engineering terms and concepts • Stepwise refinements • Some notations for structured software design • Programming language features which support software engineering design methods • Practical real-time algorithms and data structures • Real-time features of a high-level language • MODULA-2 • ADA • UNIX

273pp., 229 × 148mm, casebound
ISBN 0 86341 0162, 1984 **£22**

Orders with remittances to: IEE, PO Box 96, Stevenage, Herts. SG1 2SD, UK

Prices include postage within the UK. Outside the UK, customers should add 10% of the total price to cover postage by Bulk Air Mail to Europe. Outside Europe, 15% should be added to the price to cover postage by Accelerated Surface Post. Airmail rates are available on request. **Credit card orders (Access and Visa) are considered prepaid and will be accepted by telephone on 0438 313311.** Invoices for orders that are not prepaid will include a handling and package charge of £1.50 per book (maximum £6.00).